

МИНИСТЕРСТВО НАУКИ
И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное
образовательное учреждение высшего образования
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



НГТУ НЭТИ | Факультет прикладной математики и информатики

Кафедра прикладной математики

Практическое задание № 4

по дисциплине «Компьютерная графика»

ИНТЕРАКТИВНОЕ СОЗДАНИЕ КРИВЫХ И ПОВЕРХНОСТЕЙ С ИСПОЛЬЗОВАНИЕМ СПЛАЙНОВ

Бригада 4.5 ПМ-34 КУДЮКОВ МАКСИМ

ПМ-34 РЯБОВ ЯРОСЛАВ

Преподаватель ЗАДОРОЖНЫЙ АЛЕКСАНДР ГЕННАДЬЕВИЧ

Новосибирск, 2026

Содержание

1 Цель работы 3
2 Постановка задачи 3
3 Используемые структуры данных и функции 3
3.1 Структуры данных 3
3.2 Глобальные переменные и константы 3
3.3 Вспомогательные функции 4
3.4 Функции построения сплайна 4
3.5 Функции отрисовки 5
4 Руководство пользователя 5
4.1 Мышь 5
4.2 Клавиатура 5
4.3 Панель справа 5
5 Тестирование 6
6 Программа 9

1 Цель работы

Реализовать программу, отображающую график функции, получаемой в результате использования соответствующего сплайна.

2 Постановка задачи

1. Реализовать интерактивное задание контрольных точек мышью на плоскости.
2. Реализовать построение NURBS-кривой по контрольным точкам с учётом их весов:
    - замкнутый режим;
    - периодический режим.
3. Обеспечить автоматическое перестроение сплайна при любом изменении набора точек.
4. Реализовать навигацию по сцене:
    - перемещение камеры (ПКМ + перемещение);
    - масштабирование колесом мыши с сохранением точки под курсором;
    - отображение и подпись координатных осей и сетки.
5. При недостатке контрольных точек (меньше степень + 1) выдавать предупреждение в информационной панели и не строить сплайн.
6. Реализовать редактирование:
    - выбор и перетаскивание точки;
    - вставка точки в управляющую ломаную;
    - удаление выбранной точки и очистка всех точек;
    - изменение степени NURBS и веса выбранной точки.
7. Обеспечить отрисовку:
    - сетки с адаптивным шагом (размер ячеек в мире не «прыгает» при масштабе — меняются подписи/масштаб графика, как требует методичка);
    - координатных осей;
    - управляющего многоугольника;
    - управляющих точек (с подсветкой выбранной);
    - самой NURBS-кривой.

3 Используемые структуры данных и функции

3.1 Структуры данных

Структура	Описание
ControlPoint	Управляющая точка: X, Y, Weight (вес).
NurbsCurve	NURBS-кривая: Points, Degree, Mode, CanBuild, BuildCurve.
NurbsMode	Режим: Closed (замкнутый), Periodic (периодический).
Camera2D	2D-камера: X, Y, Zoom, преобразования экран↔мир.

3.2 Глобальные переменные и константы

Переменная / Константа	Назначение
PickRadiusPixels	Радиус захвата точки мышью (14 px).
glControl	Элемент SharpGL для вывода OpenGL.
curve	Экземпляр NURBS-кривой.
camera	Экземпляр 2D-камеры.
selectedPointIndex	Индекс выбранной точки (−1 — нет).
pointDragging, cameraDragging	Флаги перетаскивания точки/камеры.
lastMouseX, lastMouseY	Последняя позиция мыши.
degreeBox, modeBox, weightBox	UI степени, режима, веса.
infoLabel, selectedPointLabel	Информационные метки.
deleteSelectedButton, clearButton	Кнопки удаления/очистки.

3.3 Вспомогательные функции

Функция	Назначение в программе
SetupProjection()	Ортографическая проекция по камере и размерам окна.
DrawGrid(gl)	Адаптивная сетка.
CalculateGridStep()	Шаг сетки в зависимости от масштаба.
DrawAxes(gl)	Оси X и Y.
DrawControlPolygon(gl)	Управляющая ломаная (с замыканием в periodic).
DrawControlPoints(gl)	Управляющие точки с подсветкой выбранной.
DrawCurve(gl)	Построение и отрисовка NURBS-кривой.
FindPointAt(x, y)	Поиск точки под курсором.
FindControlPolygonSegment(x, y)	Поиск сегмента ломаной для вставки точки.
DistanceToSegment(...)	Расстояние от точки до отрезка.
WorldToScreenX/Y(...)	Преобразование мир → экран.
UpdateSelectedPointUI()	Обновление UI выбранной точки.
UpdateInfo()	Обновление информационной панели.
UpdateDegreeLimit(...)	Ограничение степени количеством точек.

3.4 Функции построения сплайна

Функция	Назначение
BuildCurve(samples)	Построение кривой (clamped/periodic).
BuildClamped(samples)	Замкнутая (clamped) кривая.
BuildPeriodic(samples)	Периодическая кривая с дублированием точек.
Evaluate(u, points, degree, knots)	Точка NURBS по параметру u.
Basis(i, degree, u, knots)	Базисная функция Кокса–де Бура.
CreateClampedKnotVector(...)	Clamped-вектор узлов.
CreateUniformKnotVector(...)	Равномерный вектор узлов.

3.5 Функции отрисовки

Функция	Назначение
GlControl_OpenGLInitialized(...)	Инициализация OpenGL.
GlControl_Resized(...)	Пересчёт проекции при изменении размера.
GlControl_OpenGLDraw(...)	Главная отрисовка кадра.
DrawGrid, DrawAxes, DrawControlPolygon, DrawCurve, DrawControlPoints	Отрисовка элементов сцены.

4 Руководство пользователя

Программа позволяет интерактивно создавать и редактировать NURBS-кривую на плоскости. Управление — мышью и клавиатурой, параметры кривой задаются на правой панели. Доступные действия указаны ниже.

4.1 Мышь

Действие	Назначение
ЛКМ по пустому месту	Добавить новую управляющую точку.
ЛКМ по точке	Выбрать точку.
ЛКМ по управляющей ломаной	Вставить новую точку в сегмент.
ЛКМ + drag	Перетаскивать выбранную точку.
ПКМ + drag	Перемещать камеру.
Колесо мыши	Масштабирование относительно курсора.

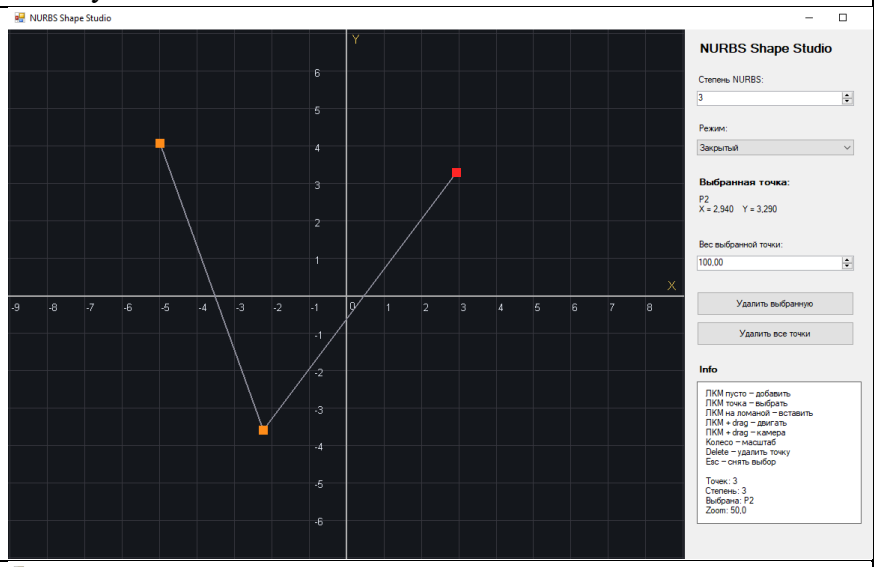
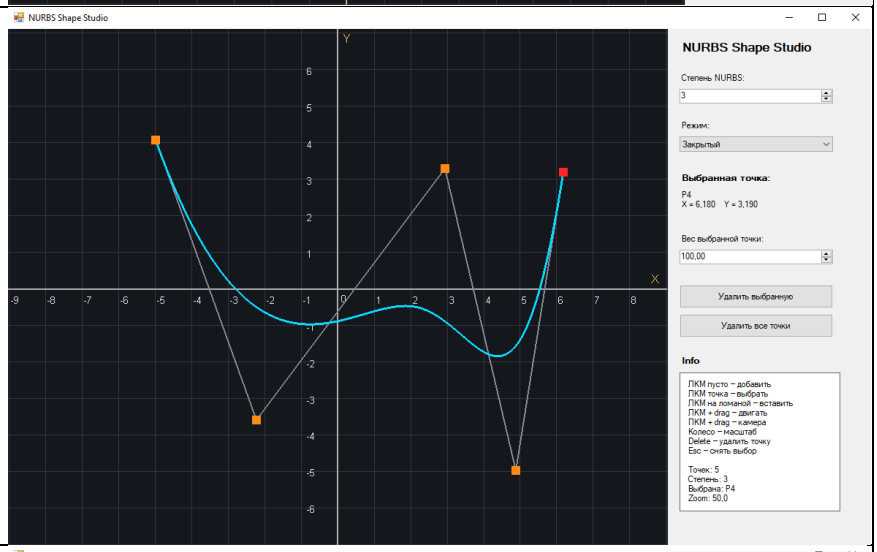
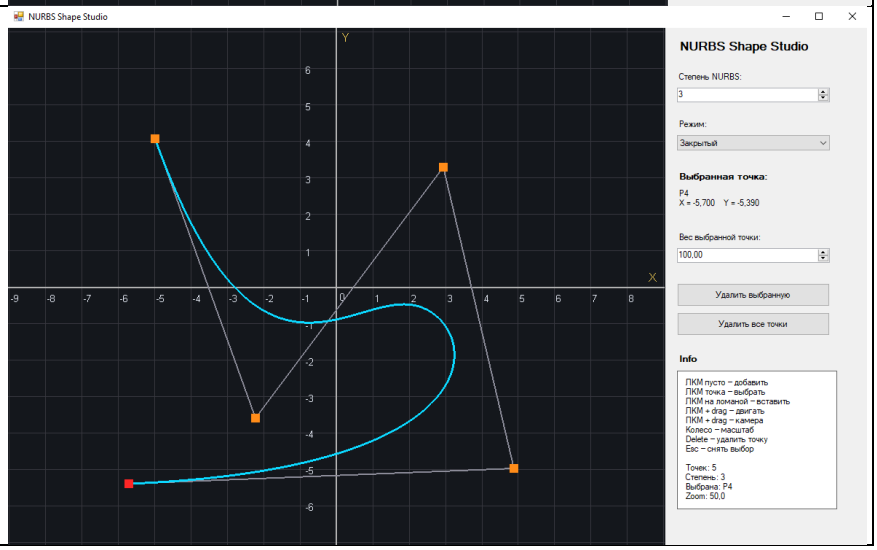
4.2 Клавиатура

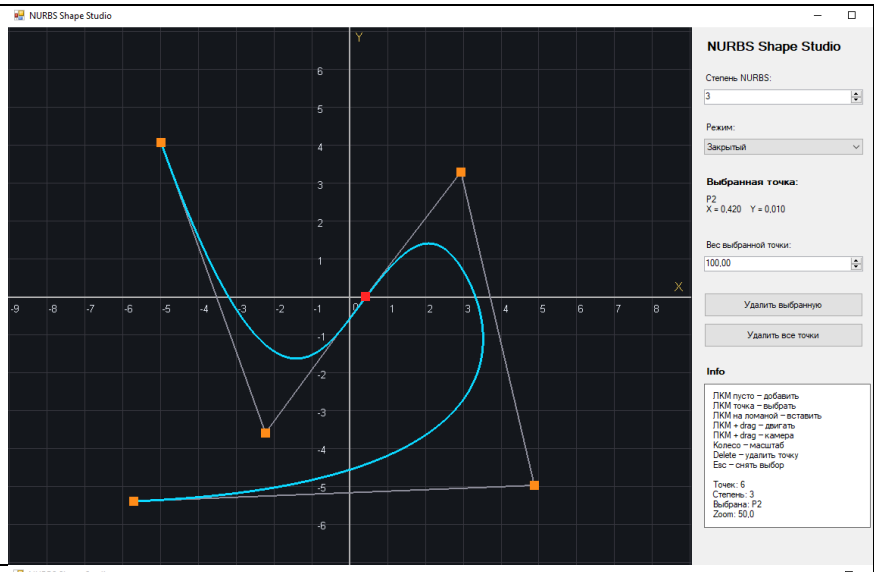
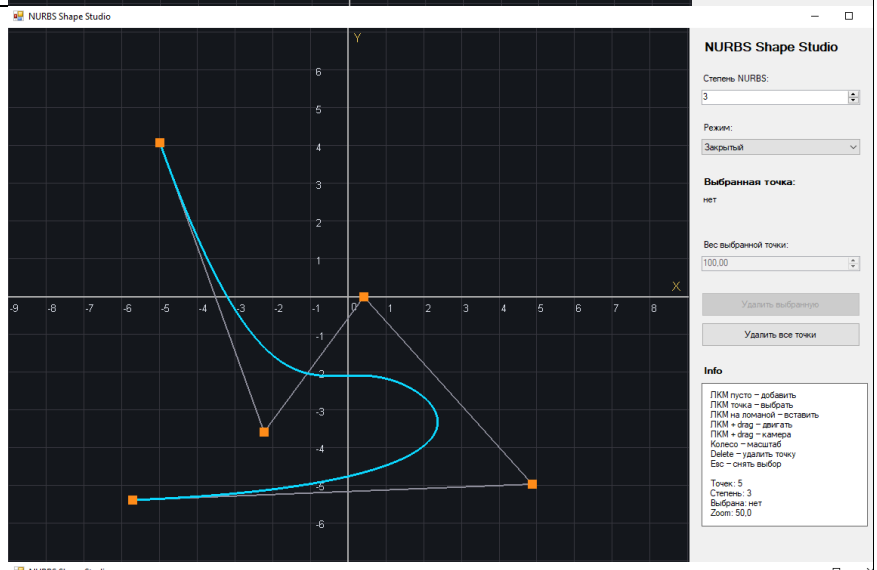
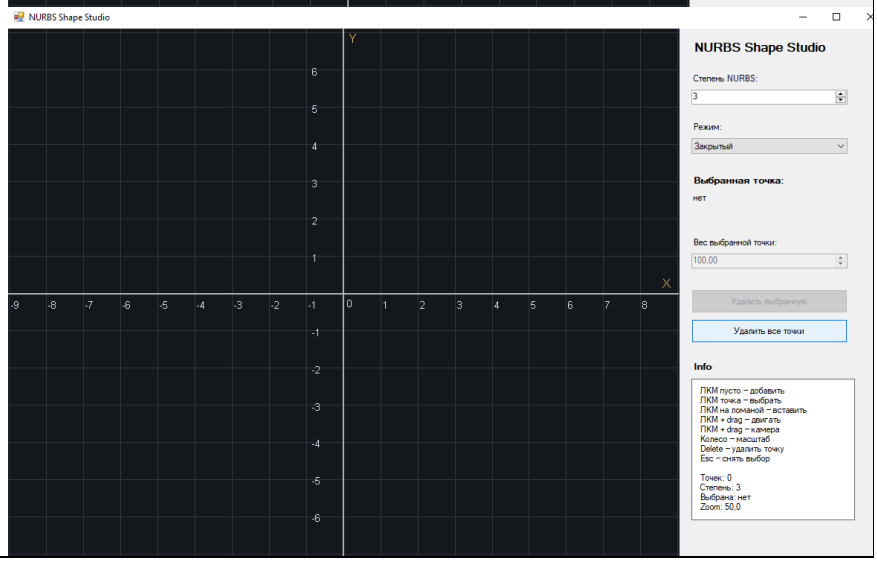
Клавиша	Назначение
Delete / Backspace	Удалить выбранную точку.
Esc	Снять выбор.

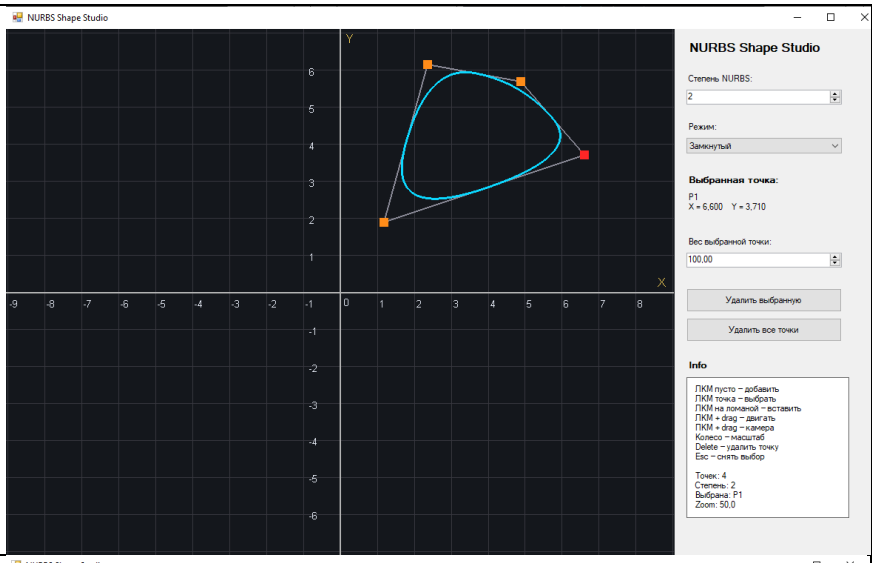
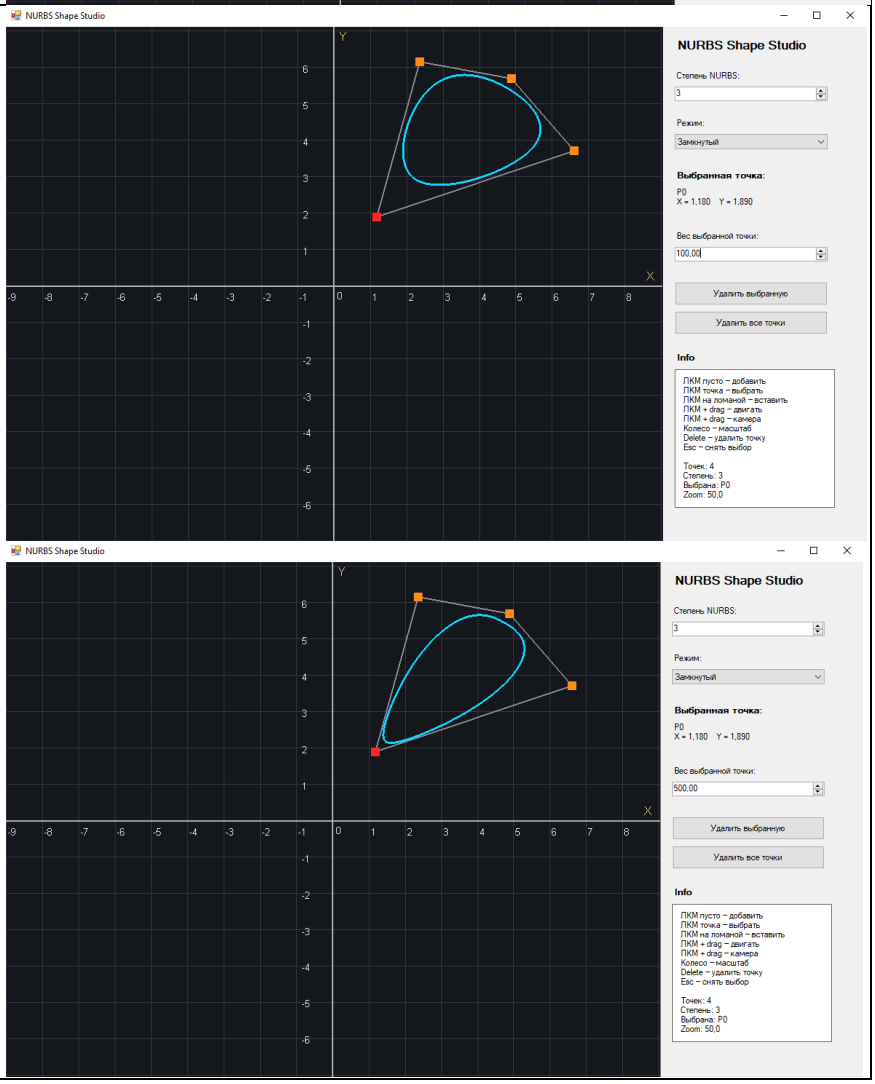
4.3 Панель справа

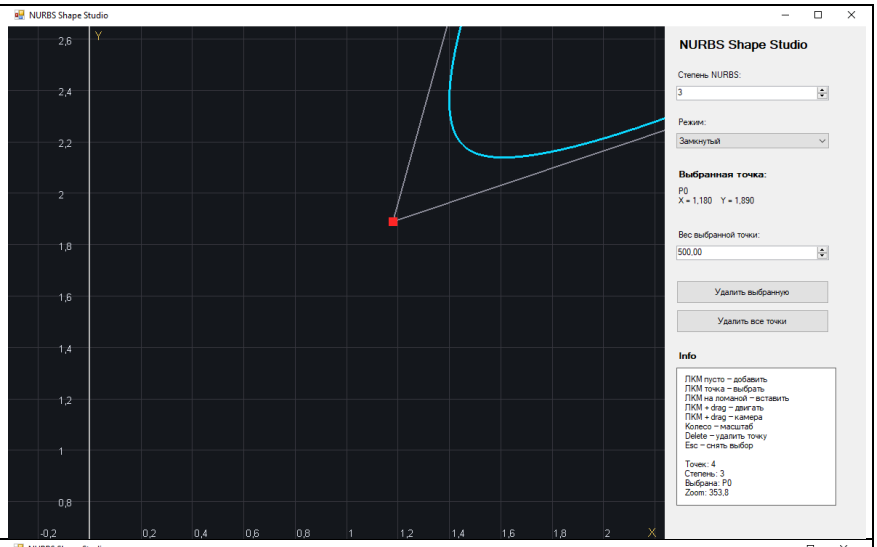
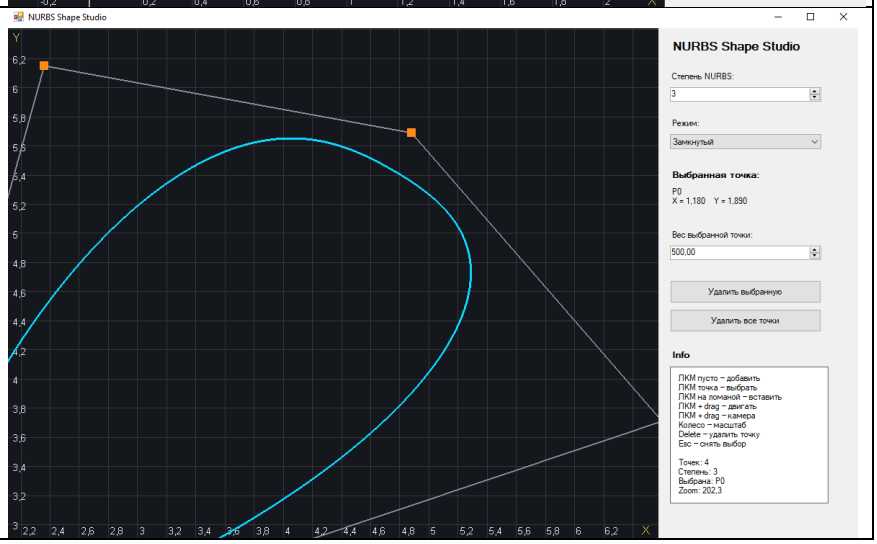
Элемент	Назначение
«Степень NURBS»	Степень кривой (1–10).
«Режим»	«Закрытый» (clamped) или «Замкнутый» (periodic).
«Вес выбранной точки»	Вес выбранной точки (значение / 100).
«Удалить выбранную»	Удалить выбранную точку.
«Удалить все точки»	Очистить все точки.
«Info»	Подсказки и текущее состояние (количество точек, степень, выбранная точка, zoom, статус построения).

5 Тестирование

№	Назначение	Результат
1	Отображение сетки, осей и управляющей ломаной	
2	Добавление управляющих точек кликом	
3	Выбор и перетаскивание точки	

4	Вставка точки в сегмент ломаной	
5	Удаление выбранной точки и очистка	 

6	Построение периодической кривой	
7	Изменение степени и веса точки	

8	Масштабирование колесом с фиксацией точки под курсором	
10	Перемещение камеры ПКМ + drag	

6 Программа

MainForm.cs — главная форма, UI, обработка мыши и клавиатуры, отрисовка сцены.

```
using System;
using System.Drawing;
using System.Windows.Forms;

using NurbsEditor.Geometry;
using NurbsEditor.Rendering;
using SharpGL;

namespace NurbsEditor
{
    public class MainForm : Form
    {
        // OpenGL / данные
        private readonly OpenGLControl glControl;
        private readonly NurbsCurve curve = new NurbsCurve();
        private readonly Camera2D camera = new Camera2D();

        // UI
        private readonly NumericUpDown degreeBox;
        private readonly ComboBox modeBox;
        private readonly Label infoLabel;
```

```
        private readonly Label selectedPointLabel;
        private readonly NumericUpDown weightBox;
        private readonly Button deleteSelectedButton;
        private readonly Button clearButton;

        // Выбор и перетаскивание
        private int selectedPointIndex = -1;
        private bool pointDragging;
        private bool cameraDragging;
        private int lastMouseX;
        private int lastMouseY;

        // Радиус захвата точки мышью в пикселях.
        private const double PickRadiusPixels = 14.0;

        // Конструктор
        public MainForm()
        {
            Text = "NURBS Shape Studio";
            Width = 1200;
            Height = 750;
            StartPosition = FormStartPosition.CenterScreen;
            KeyPreview = true;
            KeyDown += MainForm_KeyDown;

            // Главное разделение окна
            SplitContainer split = new SplitContainer();
            split.Dock = DockStyle.Fill;
            split.Orientation = Orientation.Vertical;
            split.FixedPanel = FixedPanel.Panel2;
            Controls.Add(split);

            // Теперь у split уже есть размер
            split.Panel1MinSize = 300;
            split.Panel2MinSize = 250;
            int desiredRightWidth = 280;
            int desiredDistance = split.Width - desiredRightWidth;
            int minDistance = split.Panel1MinSize;
            int maxDistance = split.Width - split.Panel2MinSize;
            split.SplitterDistance = Math.Max(minDistance, Math.Min(desiredDistance,
maxDistance));

            // OpenGLControl
            glControl = new OpenGLControl();
            ((System.ComponentModel.ISupportInitialize)glControl).BeginInit();
            glControl.Dock = DockStyle.Fill;
            glControl.OpenGLVersion = SharpGL.Version.OpenGLVersion.OpenGL2_1;
            glControl.RenderContextType = RenderContextType.DIBSection;
            glControl.FrameRate = 60;
            glControl.DrawFPS = false;
            glControl.OpenGLInitialized += GlControl_OpenGLInitialized;
            glControl.OpenGLDraw += GlControl_OpenGLDraw;
            glControl.Resized += GlControl_Resized;
            glControl.MouseDown += GlControl_MouseDown;
            glControl.MouseUp += GlControl_MouseUp;
            glControl.MouseMove += GlControl_MouseMove;
            glControl.MouseWheel += GlControl_MouseWheel;
            ((System.ComponentModel.ISupportInitialize)glControl).EndInit();
            split.Panel1.Controls.Add(glControl);

            // Правая панель
            Panel rightPanel = split.Panel2;
            rightPanel.Padding = new Padding(12);
```

```
            rightPanel.AutoScroll = true;

            // Заголовок
            Label titleLabel = new Label();
            titleLabel.Text = "NURBS Shape Studio";
            titleLabel.Font = new Font(Font.FontFamily, 12, FontStyle.Bold);
            titleLabel.AutoSize = true;
            titleLabel.Left = 12;
            titleLabel.Top = 15;
            rightPanel.Controls.Add(titleLabel);

            // Степень
            Label degreeLabel = new Label();
            degreeLabel.Text = "Степень NURBS:";
            degreeLabel.AutoSize = true;
            degreeLabel.Left = 12;
            degreeLabel.Top = 60;
            rightPanel.Controls.Add(degreeLabel);
            degreeBox = new NumericUpDown();
            degreeBox.Left = 12;
            degreeBox.Top = 82;
            degreeBox.Width = 210;
            degreeBox.Minimum = 1;
            degreeBox.Maximum = 10;
            degreeBox.Value = 3;
            degreeBox.ValueChanged += DegreeBox_ValueChanged;
            rightPanel.Controls.Add(degreeBox);

            // Режим
            Label modeLabel = new Label();
            modeLabel.Text = "Режим:";
            modeLabel.AutoSize = true;
            modeLabel.Left = 12;
            modeLabel.Top = 125;
            rightPanel.Controls.Add(modeLabel);
            modeBox = new ComboBox();
            modeBox.Left = 12;
            modeBox.Top = 147;
            modeBox.Width = 210;
            modeBox.DropDownStyle = ComboBoxStyle.DropDownList;
            modeBox.Items.Add("Закрытый");
            modeBox.Items.Add("Замкнутый");
            modeBox.SelectedIndex = 0;
            modeBox.SelectedIndexChanged += ModeBox_SelectedIndexChanged;
            rightPanel.Controls.Add(modeBox);

            // Выбранная точка
            Label selectionHeader = new Label();
            selectionHeader.Text = "Выбранная точка:";
            selectionHeader.AutoSize = true;
            selectionHeader.Font = new Font(Font.FontFamily, 9, FontStyle.Bold);
            selectionHeader.Left = 12;
            selectionHeader.Top = 195;
            rightPanel.Controls.Add(selectionHeader);
            selectedPointLabel = new Label();
            selectedPointLabel.Left = 12;
            selectedPointLabel.Top = 220;
            selectedPointLabel.Width = 220;
            selectedPointLabel.Height = 55;
            selectedPointLabel.Text = "нет";
            rightPanel.Controls.Add(selectedPointLabel);

            // Вес
```

```
            Label weightLabel = new Label();
            weightLabel.Text = "Вес выбранной точки:";
            weightLabel.AutoSize = true;
            weightLabel.Left = 12;
            weightLabel.Top = 280;
            rightPanel.Controls.Add(weightLabel);
            weightBox = new NumericUpDown();
            weightBox.Left = 12;
            weightBox.Top = 302;
            weightBox.Width = 210;
            weightBox.DecimalPlaces = 2;
            weightBox.Minimum = 1;
            weightBox.Maximum = 1000;
            weightBox.Increment = 10;

            // Здесь используем деление на 100:
            // 100 -> вес 1.00
            weightBox.Value = 100;
            weightBox.Enabled = false;
            weightBox.ValueChanged += WeightBox_ValueChanged;
            rightPanel.Controls.Add(weightBox);

            // Удалить выбранную
            deleteSelectedButton = new Button();
            deleteSelectedButton.Text = "Удалить выбранную";
            deleteSelectedButton.Left = 12;
            deleteSelectedButton.Top = 350;
            deleteSelectedButton.Width = 210;
            deleteSelectedButton.Height = 32;
            deleteSelectedButton.Enabled = false;
            deleteSelectedButton.Click += DeleteSelectedButton_Click;
            rightPanel.Controls.Add(deleteSelectedButton);

            // Очистить
            clearButton = new Button();
            clearButton.Text = "Удалить все точки";
            clearButton.Left = 12;
            clearButton.Top = 390;
            clearButton.Width = 210;
            clearButton.Height = 32;
            clearButton.Click += ClearButton_Click;
            rightPanel.Controls.Add(clearButton);

            // Info
            Label infoHeader = new Label();
            infoHeader.Text = "Info";
            infoHeader.AutoSize = true;
            infoHeader.Font = new Font(Font.FontFamily, 9, FontStyle.Bold);
            infoHeader.Left = 12;
            infoHeader.Top = 445;
            rightPanel.Controls.Add(infoHeader);
            infoLabel = new Label();
            infoLabel.Left = 12;
            infoLabel.Top = 470;
            infoLabel.Width = 220;
            infoLabel.Height = 190;
            infoLabel.BorderStyle = BorderStyle.FixedSingle;
            infoLabel.Padding = new Padding(8);
            infoLabel.BackColor = SystemColors.ControlLightLight;
            infoLabel.ForeColor = Color.Black;
            rightPanel.Controls.Add(infoLabel);
            UpdateInfo();
        }
```

```
        // OpenGL initialization
        private void GlControl_OpenGLInitialized(object sender, EventArgs e)
        {
            OpenGL gl = glControl.OpenGL;
            gl.ClearColor(0.08f, 0.09f, 0.11f, 1.0f);
            gl.Disable(OpenGL.GL_DEPTH_TEST);
            gl.Disable(OpenGL.GL_LIGHTING);
            gl.Disable(OpenGL.GL_TEXTURE_2D);
        }

        private void GlControl_Resized(object sender, EventArgs e)
        {
            SetupProjection();
        }

        // Проекция
        private void SetupProjection()
        {
            OpenGL gl = glControl.OpenGL;
            int width = Math.Max(glControl.Width, 1);
            int height = Math.Max(glControl.Height, 1);
            gl.Viewport(0, 0, width, height);
            gl.MatrixMode(OpenGL.GL_PROJECTION);
            gl.LoadIdentity();
            double halfWidth = width / (2.0 * camera.Zoom);
            double halfHeight = height / (2.0 * camera.Zoom);
            gl.Ortho(
                camera.X - halfWidth, camera.X + halfWidth,
                camera.Y - halfHeight, camera.Y + halfHeight,
                -1.0, 1.0);
            gl.MatrixMode(OpenGL.GL_MODELVIEW);
            gl.LoadIdentity();
        }

        // Главная отрисовка
        private void GlControl_OpenGLDraw(object sender, RenderEventArgs args)
        {
            OpenGL gl = glControl.OpenGL;
            gl.Clear(OpenGL.GL_COLOR_BUFFER_BIT | OpenGL.GL_DEPTH_BUFFER_BIT);
            SetupProjection();
            DrawGrid(gl);
            DrawAxes(gl);
            DrawAxisLabels(gl);
            DrawControlPolygon(gl);
            DrawCurve(gl);
            DrawControlPoints(gl);
            gl.Flush();
        }

        // Сетка
        private void DrawGrid(OpenGL gl)
        {
            int width = Math.Max(glControl.Width, 1);
            int height = Math.Max(glControl.Height, 1);
            double halfWidth = width / (2.0 * camera.Zoom);
            double halfHeight = height / (2.0 * camera.Zoom);
            double left = camera.X - halfWidth;
            double right = camera.X + halfWidth;
            double bottom = camera.Y - halfHeight;
            double top = camera.Y + halfHeight;
            double gridStep = CalculateGridStep();
            gl.Color(0.22, 0.22, 0.25);
```

```
            gl.LineWidth(1);
            gl.Begin(OpenGL.GL_LINES);
            double startX = Math.Floor(left / gridStep) * gridStep;
            for (double x = startX; x <= right; x += gridStep)
            {
                gl.Vertex(x, bottom, 0.0);
                gl.Vertex(x, top, 0.0);
            }
            double startY = Math.Floor(bottom / gridStep) * gridStep;
            for (double y = startY; y <= top; y += gridStep)
            {
                gl.Vertex(left, y, 0.0);
                gl.Vertex(right, y, 0.0);
            }
            gl.End();
        }

        private double CalculateGridStep()
        {
            double worldPerPixel = 1.0 / camera.Zoom;
            double target = worldPerPixel * 80.0;
            double power = Math.Pow(10, Math.Floor(Math.Log10(target)));
            double normalized = target / power;
            double step;
            if (normalized < 2.0)
            {
                step = 1.0;
            }
            else if (normalized < 5.0)
            {
                step = 2.0;
            }
            else
            {
                step = 5.0;
            }
            return step * power;
        }

        // Оси
        private void DrawAxes(OpenGL gl)
        {
            int width = Math.Max(glControl.Width, 1);
            int height = Math.Max(glControl.Height, 1);
            double halfWidth = width / (2.0 * camera.Zoom);
            double halfHeight = height / (2.0 * camera.Zoom);
            gl.LineWidth(2.0f);
            gl.Color(0.70, 0.70, 0.70);
            gl.Begin(OpenGL.GL_LINES);

            // Ось X
            gl.Vertex(camera.X - halfWidth, 0.0, 0.0);
            gl.Vertex(camera.X + halfWidth, 0.0, 0.0);

            // Ось Y
            gl.Vertex(0.0, camera.Y - halfHeight, 0.0);
            gl.Vertex(0.0, camera.Y + halfHeight, 0.0);
            gl.End();
        }

        private void DrawAxisLabels(OpenGL gl)
        {
            int width = Math.Max(glControl.Width, 1);
```

```
            int height = Math.Max(glControl.Height, 1);
            double halfWidth = width / (2.0 * camera.Zoom);
            double halfHeight = height / (2.0 * camera.Zoom);
            double left = camera.X - halfWidth;
            double right = camera.X + halfWidth;
            double bottom = camera.Y - halfHeight;
            double top = camera.Y + halfHeight;
            double gridStep = CalculateGridStep();

            double xAxisScreenY = WorldToScreenY(0.0);
            double xAxisGlY = height - xAxisScreenY;
            double yAxisScreenX = WorldToScreenX(0.0);
            bool xAxisVisible =
                xAxisScreenY >= 0 && xAxisScreenY <= height;
            bool yAxisVisible =
                yAxisScreenX >= 0 && yAxisScreenX <= width;


            int xLabelsY;
            if (xAxisVisible)
            {
                xLabelsY = xAxisGlY >= 24
                    ? (int)Math.Round(xAxisGlY - 20)
                    : (int)Math.Round(xAxisGlY + 6);
            }
            else
            {
                xLabelsY = 8;
            }
            xLabelsY = Math.Max(4, Math.Min(height - 16, xLabelsY));

            int yLabelsX;
            if (yAxisVisible)
            {
                yLabelsX = yAxisScreenX >= 48
                    ? (int)Math.Round(yAxisScreenX - 42)
                    : (int)Math.Round(yAxisScreenX + 8);
            }
            else
            {
                yLabelsX = 8;
            }
            yLabelsX = Math.Max(4, Math.Min(width - 42, yLabelsX));


            gl.MatrixMode(OpenGL.GL_PROJECTION);
            gl.PushMatrix();
            gl.LoadIdentity();
            gl.Ortho(0, width, 0, height, -1, 1);
            gl.MatrixMode(OpenGL.GL_MODELVIEW);
            gl.PushMatrix();
            gl.LoadIdentity();
            gl.Disable(OpenGL.GL_DEPTH_TEST);

            double startX = Math.Ceiling(left / gridStep) * gridStep;
            for (double x = startX; x <= right; x += gridStep)
            {
                int screenX = (int)Math.Round(WorldToScreenX(x));
                if (screenX < 2 || screenX > width - 42)
                    continue;

                if (Math.Abs(x) < gridStep * 0.000001)
                    continue;
```

```
                gl.DrawText(screenX + 3, xLabelsY, 0.88f, 0.90f, 0.94f, "Arial",
12f, x.ToString("0.###"));
            }


            double startY = Math.Ceiling(bottom / gridStep) * gridStep;
            for (double y = startY; y <= top; y += gridStep)
            {
                int screenY = (int)Math.Round(WorldToScreenY(y));
                if (screenY < 8 || screenY > height - 16)
                    continue;
                if (Math.Abs(y) < gridStep * 0.000001)
                    continue;
                int labelGlY = height - screenY - 6;
                gl.DrawText(yLabelsX, labelGlY, 0.88f, 0.90f, 0.94f, "Arial", 12f,
y.ToString("0.###"));
            }

            if (xAxisVisible && yAxisVisible)
            {
                int zeroX = (int)Math.Round(yAxisScreenX) + 5;
                int zeroY = (int)Math.Round(xAxisGlY) - 18;
                zeroX = Math.Max(4, Math.Min(width - 24, zeroX));
                zeroY = Math.Max(4, Math.Min(height - 16, zeroY));
                gl.DrawText(zeroX, zeroY, 0.88f, 0.90f, 0.94f, "Arial", 12f, "0");
            }

            int xNameY = xAxisVisible
                ? (int)Math.Round(xAxisGlY + 8)
                : 8;
            xNameY = Math.Max(8, Math.Min(height - 18, xNameY));
            int yNameX = yAxisVisible
                ? (int)Math.Round(yAxisScreenX + 8)
                : 8;
            yNameX = Math.Max(8, Math.Min(width - 18, yNameX));
            gl.DrawText(width - 22, xNameY, 1.0f, 0.85f, 0.35f, "Arial", 14f, "X");
            gl.DrawText(yNameX, height - 18, 1.0f, 0.85f, 0.35f, "Arial", 14f, "Y");

            gl.PopMatrix();
            gl.MatrixMode(OpenGL.GL_PROJECTION);
            gl.PopMatrix();
            gl.MatrixMode(OpenGL.GL_MODELVIEW);
        }

        // Управляющий многоугольник
        private void DrawControlPolygon(OpenGL gl)
        {
            if (curve.Points.Count < 2)
            {
                return;
            }
            gl.Color(0.55, 0.55, 0.60);
            gl.LineWidth(1.5f);
            gl.Begin(OpenGL.GL_LINE_STRIP);
            foreach (ControlPoint point in curve.Points)
            {
                gl.Vertex(point.X, point.Y, 0.0);
            }
            gl.End();

            if (curve.Mode == NurbsMode.Periodic && curve.Points.Count >= 2)
            {
                ControlPoint first = curve.Points[0];
```

```
            ControlPoint last = curve.Points[curve.Points.Count - 1];
            gl.Begin(OpenGL.GL_LINES);
            gl.Vertex(last.X, last.Y, 0.0);
            gl.Vertex(first.X, first.Y, 0.0);
            gl.End();
        }
    }

    // Контрольные точки
    private void DrawControlPoints(OpenGL gl)
    {
        if (curve.Points.Count == 0)
        {
            return;
        }
        gl.PointSize(12.0f);
        gl.Begin(OpenGL.GL_POINTS);
        for (int i = 0; i < curve.Points.Count; i++)
        {
            ControlPoint point = curve.Points[i];
            if (i == selectedPointIndex)
            {

                // Выбранная точка
                gl.Color(1.0, 0.15, 0.15);
            }
            else
            {

                // Обычная точка
                gl.Color(1.0, 0.55, 0.10);
            }
            gl.Vertex(point.X, point.Y, 0.0);
        }
        gl.End();
    }

    // NURBS
    private void DrawCurve(OpenGL gl)
    {
        if (!curve.CanBuild)
        {
            return;
        }
        var curvePoints = curve.BuildCurve(600);
        if (curvePoints.Count == 0)
        {
            return;
        }
        gl.Color(0.05, 0.85, 1.00);
        gl.LineWidth(3.0f);
        gl.Begin(OpenGL.GL_LINE_STRIP);
        foreach (PointF point in curvePoints)
        {
            gl.Vertex(point.X, point.Y, 0.0);
        }
        gl.End();
    }

    // Поиск точки под мышью
    private int FindPointAt(int mouseX, int mouseY)
    {
        int bestIndex = -1;
```

```
            double bestDistance = PickRadiusPixels;
            for (int i = 0; i < curve.Points.Count; i++)
            {
                ControlPoint p = curve.Points[i];
                double screenX = WorldToScreenX(p.X);
                double screenY = WorldToScreenY(p.Y);
                double dx = mouseX - screenX;
                double dy = mouseY - screenY;
                double distance = Math.Sqrt(dx * dx + dy * dy);
                if (distance <= bestDistance)
                {
                    bestDistance = distance;
                    bestIndex = i;
                }
            }
            return bestIndex;
        }

        private double WorldToScreenX(double worldX)
        {
            return (worldX - camera.X) * camera.Zoom + glControl.Width / 2.0;
        }

        private double WorldToScreenY(double worldY)
        {
            return
            glControl.Height / 2.0 - (worldY - camera.Y) * camera.Zoom;
        }

        // MouseDown
        private void GlControl_MouseDown(object sender, MouseEventArgs e)
        {
            glControl.Focus();
            lastMouseX = e.X;
            lastMouseY = e.Y;

            // ПКМ - камера
            if (e.Button == MouseButtons.Right)
            {
                cameraDragging = true;
                return;
            }
            if (e.Button != MouseButtons.Left) return;

            int pointIndex = FindPointAt(e.X, e.Y);
            if (pointIndex >= 0)
            {
                selectedPointIndex = pointIndex;
                pointDragging = true;
                UpdateSelectedPointUI();
                glControl.Refresh();
                return;
            }

            // Координаты клика в мире
            double x = camera.ScreenToWorldX(e.X, glControl.Width);
            double y = camera.ScreenToWorldY(e.Y, glControl.Height);

            int segment = FindControlPolygonSegment(e.X, e.Y);
            if (segment >= 0)
            {

                curve.Points.Insert(segment + 1, new ControlPoint(x, y, 1.0));
```

```
                UpdateDegreeLimit();
                selectedPointIndex = segment + 1;
                pointDragging = true;
                UpdateSelectedPointUI();
                UpdateInfo();
                glControl.Refresh();
                return;
            }

            curve.Points.Add(new ControlPoint(x, y, 1.0));
            UpdateDegreeLimit();
            selectedPointIndex = curve.Points.Count - 1;
            pointDragging = true;
            UpdateSelectedPointUI();
            UpdateInfo();
            glControl.Refresh();
        }

        // MouseMove
        private void GlControl_MouseMove(object sender, MouseEventArgs e)
        {

            // Перетаскивание контрольной точки
            if (pointDragging && selectedPointIndex >= 0 && selectedPointIndex <
curve.Points.Count)
            {
                double x = camera.ScreenToWorldX(e.X, glControl.Width);
                double y = camera.ScreenToWorldY(e.Y, glControl.Height);
                ControlPoint point = curve.Points[selectedPointIndex];
                point.X = x;
                point.Y = y;
                UpdateSelectedPointUI();
                UpdateInfo();
                glControl.Refresh();
                return;
            }

            // Перемещение камеры
            if (cameraDragging)
            {
                int dx = e.X - lastMouseX;
                int dy = e.Y - lastMouseY;
                camera.X -= dx / camera.Zoom;
                camera.Y += dy / camera.Zoom;
                lastMouseX = e.X;
                lastMouseY = e.Y;
                glControl.Refresh();
            }
        }

        // MouseUp
        private void GlControl_MouseUp(object sender, MouseEventArgs e)
        {
            if (e.Button == MouseButtons.Left)
            {
                pointDragging = false;
            }
            if (e.Button == MouseButtons.Right)
            {
                cameraDragging = false;
            }
        }
```

```
        // Zoom
        private void GlControl_MouseWheel(object sender, MouseEventArgs e)
        {

            double oldWorldX = camera.ScreenToWorldX(e.X, glControl.Width);
            double oldWorldY = camera.ScreenToWorldY(e.Y, glControl.Height);
            if (e.Delta > 0)
            {
                camera.Zoom *= 1.15;
            }
            else
            {
                camera.Zoom /= 1.15;
            }
            camera.Zoom = Math.Max(5.0, Math.Min(1000.0, camera.Zoom));

            double newWorldX = camera.ScreenToWorldX(e.X, glControl.Width);
            double newWorldY = camera.ScreenToWorldY(e.Y, glControl.Height);

            // Масштабирование относительно курсора.
            camera.X += oldWorldX - newWorldX;
            camera.Y += oldWorldY - newWorldY;
            UpdateInfo();
            glControl.Refresh();
        }

        // Степень
        private void DegreeBox_ValueChanged(object sender, EventArgs e)
        {
            curve.Degree = (int)degreeBox.Value;
            UpdateInfo();
            glControl.Refresh();
        }

        // Ограничиваем степень количеством контрольных точек.
        // До добавления точек разрешаем выбрать степень заранее.
        private void UpdateDegreeLimit(bool pointsRemoved = false)
        {
            int maxDegree;
            if (curve.Points.Count == 0)
            {

                // Пользователь может заранее настроить степень.
                maxDegree = 10;
            }
            else if (pointsRemoved)
            {

                maxDegree = Math.Max(1, curve.Points.Count - 1);
            }
            else
            {

                maxDegree = Math.Max(curve.Degree, Math.Max(1, curve.Points.Count -
1));
            }
            maxDegree = Math.Min(10, maxDegree);

            if (degreeBox.Value > maxDegree)
            {
                degreeBox.Value = maxDegree;
            }
            degreeBox.Maximum = maxDegree;
```

```
            curve.Degree = (int)degreeBox.Value;
        }

        // Режим
        private void ModeBox_SelectedIndexChanged(object sender, EventArgs e)
        {
            curve.Mode = modeBox.SelectedIndex == 0 ? NurbsMode.Closed :
NurbsMode.Periodic;
            UpdateInfo();
            glControl.Refresh();
        }

        // Вес точки
        private void WeightBox_ValueChanged(object sender, EventArgs e)
        {
            if (selectedPointIndex < 0 || selectedPointIndex >= curve.Points.Count)
            {
                return;
            }
            double weight = (double)weightBox.Value / 100.0;
            curve.Points[selectedPointIndex].Weight = weight;
            UpdateSelectedPointUI();
            UpdateInfo();
            glControl.Refresh();
        }

        // Delete выбранной точки
        private void DeleteSelectedPoint()
        {
            if (selectedPointIndex < 0 || selectedPointIndex >= curve.Points.Count)
            {
                return;
            }
            curve.Points.RemoveAt(selectedPointIndex);
            UpdateDegreeLimit(pointsRemoved: true);
            selectedPointIndex = -1;
            pointDragging = false;
            UpdateSelectedPointUI();
            UpdateInfo();
            glControl.Refresh();
        }

        private void DeleteSelectedButton_Click(object sender, EventArgs e)
        {
            DeleteSelectedPoint();
        }

        // Очистка
        private void ClearButton_Click(object sender, EventArgs e)
        {
            curve.Points.Clear();
            UpdateDegreeLimit();
            selectedPointIndex = -1;
            pointDragging = false;
            UpdateSelectedPointUI();
            UpdateInfo();
            glControl.Refresh();
        }

        // Клавиатура
        private void MainForm_KeyDown(object sender, KeyEventArgs e)
        {
            if (e.KeyCode == Keys.Delete || e.KeyCode == Keys.Back)
```

```
            {
                DeleteSelectedPoint();
                e.Handled = true;
                return;
            }
            if (e.KeyCode == Keys.Escape)
            {
                selectedPointIndex = -1;
                pointDragging = false;
                UpdateSelectedPointUI();
                UpdateInfo();
                glControl.Refresh();
            }
        }

        private void UpdateSelectedPointUI()
        {
            bool hasSelection = selectedPointIndex >= 0 && selectedPointIndex <
curve.Points.Count;
            deleteSelectedButton.Enabled = hasSelection;
            weightBox.Enabled = hasSelection;
            if (!hasSelection)
            {
                selectedPointLabel.Text = "нет";
                return;
            }
            ControlPoint p = curve.Points[selectedPointIndex];
            selectedPointLabel.Text = string.Format(
                "P{0}\nX = {1:0.000}    Y = {2:0.000}",
                selectedPointIndex, p.X, p.Y);
            decimal weightValue = (decimal)(p.Weight * 100.0);
            if (weightValue < weightBox.Minimum)
            {
                weightValue = weightBox.Minimum;
            }
            if (weightValue > weightBox.Maximum)
            {
                weightValue = weightBox.Maximum;
            }

            if (weightBox.Value != weightValue)
            {
                weightBox.Value = weightValue;
            }
        }

        // Info
        private void UpdateInfo()
        {
            string status;
            if (curve.CanBuild)
            {
                status = "Сплайн построен";
            }
            else
            {
                status = "Недостаточно точек.\n" + "Нужно минимум: " + (curve.Degree
+ 1);
            }
            string selected;
            if (selectedPointIndex >= 0 && selectedPointIndex < curve.Points.Count)
            {
                selected = "P" + selectedPointIndex;
```

```
            }
            else
            {
                selected = "нет";
            }
            infoLabel.Text =
                "ЛКМ пусто — добавить\n" +
                "ЛКМ точка — выбрать\n" +
                "ЛКМ на ломаной — вставить\n" +
                "ЛКМ + drag — двигать\n" +
                "ПКМ + drag — камера\n" +
                "Колесо — масштаб\n" +
                "Delete — удалить точку\n" +
                "Esc — снять выбор\n\n" +
                string.Format("Точек: {0}\n", curve.Points.Count) +
                string.Format("Степень: {0}\n", curve.Degree) +
                string.Format("Выбрана: {0}\n", selected) +
                string.Format("Zoom: {0:0.0}\n\n", camera.Zoom) +
                status;
        }

        private int FindControlPolygonSegment(int mouseX, int mouseY)
        {
            if (curve.Points.Count < 2) return -1;
            double minDistance = 10.0;
            int result = -1;
            for (int i = 0; i < curve.Points.Count - 1; i++)
            {
                ControlPoint a = curve.Points[i];
                ControlPoint b = curve.Points[i + 1];
                double ax = WorldToScreenX(a.X);
                double ay = WorldToScreenY(a.Y);
                double bx = WorldToScreenX(b.X);
                double by = WorldToScreenY(b.Y);
                double distance = DistanceToSegment(mouseX, mouseY, ax, ay, bx, by);
                if (distance < minDistance)
                {
                    minDistance = distance;
                    result = i;
                }
            }

            if (curve.Mode == NurbsMode.Periodic)
            {
                ControlPoint a = curve.Points[curve.Points.Count - 1];
                ControlPoint b = curve.Points[0];
                double distance = DistanceToSegment(
                    mouseX, mouseY,
                    WorldToScreenX(a.X), WorldToScreenY(a.Y),
                    WorldToScreenX(b.X), WorldToScreenY(b.Y));
                if (distance < minDistance)
                {
                    result = curve.Points.Count - 1;
                }
            }
            return result;
        }

        private double DistanceToSegment(double px, double py, double x1, double y1,
double x2, double y2)
        {
            double dx = x2 - x1;
            double dy = y2 - y1;
```

```
            if (dx == 0 && dy == 0)
                return Math.Sqrt((px - x1) * (px - x1) + (py - y1) * (py - y1));
            double t = ((px - x1) * dx + (py - y1) * dy) / (dx * dx + dy * dy);
            t = Math.Max(0, Math.Min(1, t));
            double nx = x1 + t * dx;
            double ny = y1 + t * dy;
            return Math.Sqrt((px - nx) * (px - nx) + (py - ny) * (py - ny));
        }
    }
}
```

NurbsCurve.cs — построение NURBS-кривой

```
using System;
using System.Collections.Generic;
using System.Drawing;

namespace NurbsEditor.Geometry
{
    public enum NurbsMode
    {
        Closed,
        Periodic
    }

    public class NurbsCurve
    {
        public List<ControlPoint> Points
        {
            get;
        } = new List<ControlPoint>();

        public int Degree
        {
            get;
            set;
        } = 3;

        public NurbsMode Mode
        {
            get;
            set;
        } = NurbsMode.Closed;

        public bool CanBuild
        {
            get
            {
                return Points.Count >= Degree + 1;
            }
        }

        public List<PointF> BuildCurve(int samples = 400)
        {
            List<PointF> result = new List<PointF>();

            if (!CanBuild) return result;

            if (Mode == NurbsMode.Closed) return BuildClamped(samples);

            return BuildPeriodic(samples);
        }

        // ----------------------------------------
```

```
        // Закрытый / закреплённый вариант
        // ----------------------------------------

        private List<PointF> BuildClamped(int samples)
        {
            List<PointF> result = new List<PointF>();

            int n = Points.Count - 1;
            int p = Degree;

            double[] knots = CreateClampedKnotVector(Points.Count, p);

            double uMin = knots[p];
            double uMax = knots[n + 1];

            for (int s = 0; s <= samples; s++)
            {
                double t = (double)s / samples;

                double u = uMin + (uMax - uMin) * t;

                // Для самой последней точки
                // чуть уменьшаем параметр.
                if (s == samples) u = uMax - 1e-10;

                PointF point = Evaluate(u, Points, p, knots);

                result.Add(point);
            }

            return result;
        }

        // ----------------------------------------
        // Периодический / замкнутый вариант
        // ----------------------------------------

        private List<PointF> BuildPeriodic(int samples)
        {
            List<ControlPoint> periodicPoints = new List<ControlPoint>();

            foreach (ControlPoint p in Points)
            {
                periodicPoints.Add(new ControlPoint(p.X, p.Y, p.Weight));
            }

            // Добавляем первые Degree точек
            // после конца массива.
            for (int i = 0; i < Degree; i++)
            {
                ControlPoint p = Points[i % Points.Count];

                periodicPoints.Add(new ControlPoint(p.X, p.Y, p.Weight));
            }

            int pDegree = Degree;

            double[] knots = CreateUniformKnotVector(periodicPoints.Count, pDegree);

            int n = periodicPoints.Count - 1;

            double uMin = knots[pDegree];
            double uMax = knots[n + 1];
```

```
            // Для полного замыкания берем диапазон,
            // соответствующий исходному числу точек.
            double periodEnd = uMin + Points.Count;

            if (periodEnd < uMax) uMax = periodEnd;

            List<PointF> result = new List<PointF>();

            for (int s = 0; s <= samples; s++)
            {
                double t = (double)s / samples;

                double u = uMin + (uMax - uMin) * t;

                if (s == samples) u = uMax - 1e-10;

                result.Add(Evaluate(u, periodicPoints, pDegree, knots));
            }

            // визуально замыкаем линию
            if (result.Count > 0) result.Add(result[0]);

            return result;
        }

        // -----------------------------------------
        // NURBS
        // -----------------------------------------

        private PointF Evaluate(
        double u, List<ControlPoint> points, int degree, double[] knots)
        {
            double numeratorX = 0.0;
            double numeratorY = 0.0;
            double denominator = 0.0;

            for (int i = 0; i < points.Count; i++)
            {
                double basis = Basis(i, degree, u, knots);

                double value = basis * points[i].Weight;

                numeratorX += value * points[i].X;

                numeratorY += value * points[i].Y;

                denominator += value;
            }

            if (Math.Abs(denominator) < 1e-12) return PointF.Empty;

            return new PointF((float)(numeratorX / denominator), (float)(numeratorY
/ denominator));
        }

        // Формула Кокса-де Бура
        private double Basis(
        int i, int degree, double u, double[] knots)
        {
            if (degree == 0)
            {
                if (knots[i] <= u && u < knots[i + 1]) return 1.0;
```

```
                return 0.0;
            }

            double left = 0.0;
            double right = 0.0;

            double leftDenominator = knots[i + degree] - knots[i];

            if (Math.Abs(leftDenominator) > 1e-12)
            {
                left = (u - knots[i]) / leftDenominator * Basis(i, degree - 1, u,
knots);
            }

            double rightDenominator = knots[i + degree + 1] - knots[i + 1];

            if (Math.Abs(rightDenominator) > 1e-12)
            {
                right = (knots[i + degree + 1] - u) / rightDenominator * Basis(i +
1, degree - 1, u, knots);
            }

            return left + right;
        }

        // ------------------------------------------
        // Узловые векторы
        // ------------------------------------------

        private double[] CreateClampedKnotVector(
        int controlPointCount, int degree)
        {
            int n = controlPointCount - 1;

            int knotCount = n + degree + 2;

            double[] knots = new double[knotCount];

            int maxValue = n - degree + 1;

            for (int i = 0; i < knotCount; i++)
            {
                if (i <= degree)
                {
                    knots[i] = 0;
                }
                else if (i >= n + 1)
                {
                    knots[i] = maxValue;
                }
                else
                {
                    knots[i] = i - degree;
                }
            }

            return knots;
        }

        private double[] CreateUniformKnotVector(
        int controlPointCount, int degree)
        {
```

```
            int knotCount = controlPointCount + degree + 1;

            double[] knots = new double[knotCount];

            for (int i = 0; i < knotCount; i++)
                knots[i] = i;

            return knots;
        }
    }
}
```

ControlPoint.cs — модель управляющей точки.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace NurbsEditor.Geometry
{
    public class ControlPoint
    {
        public double X { get; set; }
        public double Y { get; set; }

        public double Weight { get; set; }

        public ControlPoint(double x, double y, double weight = 1.0)
        {
            X = x;
            Y = y;
            Weight = weight;
        }
    }
}
```

Camera2D.cs — 2D-камера и преобразования координат.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace NurbsEditor.Rendering
{
    public class Camera2D
    {
        public double X
        {
            get;
            set;
        } = 0.0;
        public double Y
        {
            get;
            set;
        } = 0.0;

        public double Zoom
        {
            get;
            set;
        } = 50.0;
```

```
        public double ScreenToWorldX(
        int screenX, int width)
        {
            return (screenX - width / 2.0) / Zoom + X;
        }

        public double ScreenToWorldY(
        int screenY, int height)
        {
            return (height / 2.0 - screenY) / Zoom + Y;
        }
    }
}
```